The Journal of Open Source Software

anvay: a web-based tool for interpretive topic modelling in bengali

Vinayak Das Gupta [1]

1 Shiv Nadar Institute of Eminence

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor:

Submitted: 22 April 2025
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

DRAFT

Summary

anvay is a web-based tool for topic modelling in bengali, developed for exploratory reading and interpretive analysis. It provides a full pipeline for Latent Dirichlet Allocation (LDA) (Blei et al., 2003)—from corpus ingestion and preprocessing to model configuration and visual output—within a lightweight, browser-based interface. The tool foregrounds user interpretation: rather than providing coherence scores or fixed topic labels, *anvay* presents model results as provisional surfaces to be read, questioned, and restructured by the user.

Designed for literary, journalistic, and historical corpora in bengali, *anvay* supports a range of language-specific preprocessing functions including lemmatisation, frequency filtering, and stop-word pruning. The outputs—ranging from topic-word networks to document-level previews—are rendered with clarity and designed to scaffold close reading. Each topic is accessible through multiple lenses: top words, paragraph-level examples, document weights, and corpus-wide distribution.

Beyond its technical function, *anvay* is an intervention in how we teach and understand computational methods in low-resource contexts. It centres the reader, not the algorithm. Its value lies not in automation, but in the space it creates for interpretation, curation, and the possibility of collaborative meaning-making.

The source code and full documentation are available at: https://github.com/vinayakdas-gupta/anvay

Statement of Need

There are few interpretive modelling tools for bengali, and fewer still that support pedagogical engagement. Most existing frameworks for LDA assume technical fluency, rely on scripts or notebooks, and provide minimal attention to interpretability. While backend libraries like Gensim (Řehůřek & Sojka, 2010) and Mallet offer robust implementations, they do not assist users in making sense of what a topic is, how it shifts across documents, or how it might relate to thematic or conceptual questions.

anvay addresses this gap by making topic modelling visible and usable. It offers an in-browser interface with no coding requirements, and is particularly suited to students, independent researchers, and archival practitioners working in South Asian languages. It allows users to upload their own text corpora, adjust model parameters, and inspect topic boundaries through interactive, multi-modal visualisations. The tool has already been used in classroom and workshop settings to introduce interpretive topic modelling to humanities students with no programming background.

Functionality

Built in Python using Flask, Gensim, and standard data visualisation libraries, *anvay* provides the following core functionalities:

- **Corpus upload and preprocessing**: Users may upload up to 800 `.txt` files in a single run. Preprocessing options include standard and custom stopword filtering, optional stemming or lemmatisation, and frequency-based token pruning.
- **Topic model training**: Users can adjust passes, iterations, alpha, eta, chunk size, and other hyperparameters. Models are trained on the server and stored temporarily for display.
- **Language-specific processing**: Bengali tokenisation is handled with custom rules to prevent malformed segmentation. Lemma support is drawn from publicly available datasets (Alam et al., 2021; Chakrabarty et al., 2017).
- **Visual outputs**: Model results are rendered via multiple interactive modules:
  - PCA-reduced topic plots (Plotly)
  - Topic-word heatmaps (Seaborn)
  - Topic-document bar charts
  - Topic prevalence pie charts
  - Topic-word network graphs (NetworkX)
- **Interpretive scaffolds**: Users can view representative paragraphs per topic, locate dominant topics in specific files, and inspect topic salience across documents. Topics marked as incoherent or noisy receive a "Low Confidence" warning.
- **Accessibility and export**: The tool runs in modern browsers, with a responsive layout and dark mode. Results can be downloaded in `.csv` and `.txt` formats.

Research and Pedagogical Use

The design of *anvay* is informed by research-led teaching practice. Topic modelling, while widely adopted in digital humanities, often remains inaccessible due to steep learning curves and underdeveloped interfaces. *anvay* was developed to lower these barriers and support new modes of engagement with bengali textual corpora—especially where existing NLP tools fail to account for morphological variance, informal orthographies, or the diversity of textual registers in bengali.

In pedagogical contexts, *anvay* functions as a conceptual primer. It prompts students to ask: What is a topic? What assumptions shape a model's output? How do visualisations shape interpretation? The tool has been tested in classroom environments with undergraduate and postgraduate students, many of whom were engaging with topic modelling for the first time. The feedback has been consistent: the visual design, language support, and document-level previews help to render the model's assumptions legible.

To support this, *anvay* includes extensive web-based documentation—not just technical, but interpretive. Each section guides users through corpus preparation, parameter tuning, and result analysis, with annotated examples and embedded visual references. The documentation foregrounds conceptual understanding: users are encouraged to read models critically, experiment with settings, and reflect on how computational structure intersects with thematic interpretation. It is embedded directly in the interface and designed for both classroom and independent study.

Comparison with Existing Tools

Most open-source topic modelling tools prioritise scale or coherence evaluation. *pyLDAvis* (Sievert & Shirley, 2014), for instance, provides excellent topic distance visualisations and

relevance-based ranking, but relies on pre-tokenised, preprocessed input and assumes English-language conventions. It offers no support for Indic scripts, and its interface—though informative—is not designed for novice users or pedagogical contexts. Similarly, Voyant Tools (Sinclair & Rockwell, 2016) provides a visually rich environment for text exploration, but lacks custom LDA configurability and does not support Bengali tokenisation or lemmatisation.

anvay differs in both scope and ethos. It is built for **interpretation over evaluation**, and for **accessibility over scale**. Unlike pyLDAvis, which emphasises statistical overlap, *anvay* foregrounds **contextual previews**, **representative sentences**, and **visual topic-document mappings**—elements designed to support **reading**, not just inspection. Unlike Voyant, which is multilingual but limited in model customisation, *anvay* allows fine-tuning of LDA parameters, stopword filters, and token thresholds—all with support for Bengali scripts and morphologies. These choices reflect the tool's commitment to low-resource contexts, and its role as both a scholarly and pedagogical infrastructure.

Infrastructure and Development

The application is written in Python 3.9 and uses:

- **Flask** for web framework
- **Gensim** for topic modelling
- **NLTK** for tokenisation
- **Plotly**, **Seaborn**, **NetworkX** (Hagberg et al., 2008), **Bokeh** for visualisation
- **Bootstrap** and custom CSS for UI layout

Deployment is intended to be production-ready using Gunicorn, with Docker support under active development. While *anvay* handles concurrent sessions for moderate use, high-load queueing (e.g., via Celery or Redis) is not currently implemented but remains a planned feature.

Scikit-learn utilities were used for model evaluation and dimensionality support (**?**).

Performance and Limitations

anvay is designed for moderate-scale corpora, where interpretability and visual exploration are prioritised over throughput. In a benchmark run using **800 Bengali `.txt` files** (totalling **21.9MB**, ~**940,000 tokens**, and **171,754 unique vocabulary terms**), the system successfully trained a 10-topic LDA model with **10 passes** and **50 iterations** in approximately **62 seconds** on a single-core setup (Windows 10, Python 3.9, Gensim 4.3.3). This corpus included highly variable document lengths—from **79** to **86,099 tokens** per file—demonstrating robustness against input heterogeneity.

No vocabulary truncation was applied: all tokens passing frequency thresholds were retained. Earlier runs using Gensim's default settings had imposed a silent 100,000-token cap; this was resolved by explicitly setting `keep_n=None` during dictionary filtering.

While the system is tuned for formal Bengali prose, there are limitations: - **Informal or dialectal orthographies** may lead to malformed tokens - **OCR artefacts or non-Unicode glyphs** may interfere with tokenisation - The lemmatisation approach is lightweight and does not include syntactic disambiguation

The modelling backend is standard LDA; no coherence optimisation or neural alignment is included. As such, *anvay* is best used as an exploratory interface—for interpretive reading rather than automated evaluation.

DRAFT

Repository and License

The source code and documentation for *anvay* are hosted on GitHub: https://github.com/vinayak-dasgupta/anvay
The software is released under the MIT License.

Acknowledgements

anvay was developed as part of my regular teaching and research activities, without grant funding. The lemma module draws on publicly available datasets and is documented separately. Thanks are due to the developers of the open-source libraries used throughout the project. Feedback from my wife and parents has shaped both the interface logic and the project's ethos of generosity and care. ChatGPT was used to generate HTML templates, assist with interface logic, layout design, error testing, and portions of the documentation.

References

Alam, F., Hasan, Md. A., Alam, T., Khan, A., Tajrin, J., Khan, N., & Chowdhury, S. A. (2021). A review of bangla natural language processing tasks and the utility of transformer models. *arXiv Preprint arXiv:2107.03844*. https://arxiv.org/abs/2107.03844

Blei, D. M., Ng, A. Y., & Jordan, M. I. (2003). Latent dirichlet allocation. *Journal of Machine Learning Research*, *3*, 993–1022.

Chakrabarty, A., Pandit, O. P., & Garain, U. (2017). Context sensitive lemmatization using two successive bidirectional gated recurrent networks. *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics*, 1481–1491. https://doi.org/10.18653/v1/P17-1136

Hagberg, A., Swart, P., & Schult, D. A. (2008). Exploring network structure, dynamics, and function using NetworkX. *Proceedings of the 7th Python in Science Conference (SciPy)*.

Řehůřek, R., & Sojka, P. (2010). Software framework for topic modelling with large corpora. *Proceedings of the LREC 2010 Workshop on New Challenges for NLP Frameworks*.

Sievert, C., & Shirley, K. (2014). pyLDAvis: Interactive topic model visualization. *ICML Workshop on Topic Models*.

Sinclair, S., & Rockwell, G. (2016). *Voyant tools*. https://voyant-tools.org/.

DRAFT